

31. break et continue : sortir, ou sauter un tour

On n'est pas obligé de finir tous les tours : on sort dès qu'on a trouvé.

1. Ce qu'il faut comprendre

`break;` **sort** de la boucle immédiatement, sans finir le tour ni faire les suivants. C'est l'outil de la **recherche** : on parcourt jusqu'à trouver, puis on s'arrête.

Ici, on cherche la **première case vide** d'une rangée — là où un jeu poserait une nouvelle pièce, un nouvel ennemi. `lire(colonne, ligne)` rend le numéro de la tuile affichée ; l'espace est la tuile **0**. Dès qu'on la trouve : on retient la colonne, et `break`.

`tours` le prouve : la boucle prévoyait 20 tours, elle n'en a fait que **5**. Sur une grande grille, s'arrêter tôt, c'est du temps de gagné — et sur une console à 4 MHz, le temps est compté.

`continue;` fait l'inverse : il **abandonne ce tour-ci** et passe **au suivant**. Ici, il saute les multiples de 3 (`i % 3 == 0`) : 0, 3, 6 et 9 ne sont pas écrits.

Le `libre = 255` de départ est une **valeur témoin** : si la rangée était pleine, la boucle finirait sans rien trouver, et 255 le dirait (aucune colonne ne porte ce numéro).

À toi : remplis toute la rangée de # et vérifie que l'écran affiche 255.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **901 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	190 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← lire(), nombre() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Reste	10 o	← main() — le reste d'une division
CopierTuiles	94 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran

```

Tuiles          384 o ← CopierTuiles - 24 lettres sur 43
"#### ## ####..." 20 o ← texte() - un texte gravé
"CASE LIBRE:"       11 o ← texte() - un texte gravé
"TOURS FAITS:"      12 o ← texte() - un texte gravé
Lettres gravées : ABCEFILORSTU0123456789:#
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert,
Notes
Fonctions du langage appelées :
nombre() × 3      42 o sur place, dans main() + AdresseCase (10 o), EcrireNombre (33 o)
texte() × 3       33 o sur place, dans main() + EcrireTexte (10 o)
ecran() × 2       21 o sur place, dans main() + AttendreVBlank (16 o)
lire() × 1        11 o sur place, dans main() + AdresseCase (10 o)
image() × 1        3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : cherche la première case vide d'une ligne (avec break),
// puis écrit des chiffres en sautant les multiples de 3 (avec continue).
//
// Ce qui est nouveau :
// break = sortir de la boucle TOUT DE SUITE, sans finir les tours.
// continue = sauter la FIN de ce tour, et passer au tour suivant.

#include <ecran> // éteint ou rallume l'écran
#include <texte> // écrit un texte à l'écran
#include <lire>  // lit la tuile posée sur une case
#include <nombre> // écrit un nombre en chiffres
#include <reste> // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair

```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() { // Le jeu commence ici.
    ecran(0); // Dessin écran éteint.
    texte(0, 3, "#### ## #####"); // Une ligne avec des trous :
                                    // colonnes 0 à 3 = #, 4 et 5 = vides, ...

    uint8_t libre = 255; // 255 : pas encore trouvé (une valeur « impossible »)
    uint8_t tours = 0;   // Combien de cases on a regardées.

    for (uint8_t colonne = 0; colonne < 20; colonne++) { // colonne = 0 à 19
        tours++; // une case de plus regardée
        if (lire(colonne, 3) == 0) { // lire(colonne, ligne) rend ce qui est écrit
                                    // dans la case ; 0 veut dire « vide ».
            libre = colonne; // On note la colonne vide...
            break; // trouvé : inutile de continuer
        }
    }
    // Déroulé : colonnes 0, 1, 2, 3 : un # ; colonne 4 : vide → libre = 4, break.
    // tours vaut 5 : on a regardé 5 cases au lieu de 20.

    texte(0, 5, "CASE LIBRE:"); nombre(14, 5, libre); // affiche 004
    texte(0, 6, "TOURS FAITS:"); nombre(14, 6, tours); // affiche 005

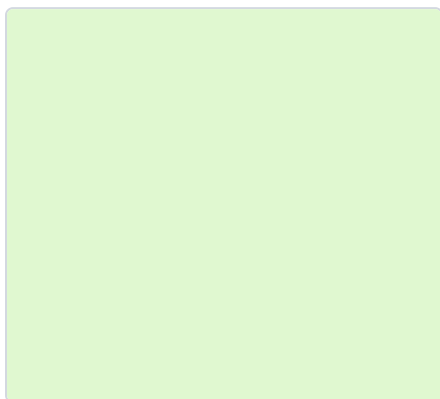
    // continue : on saute les multiples de 3.
    for (uint8_t i = 0; i < 10; i++) { // i = 0 à 9
        if (i % 3 == 0) continue; // reste de i par 3 égal à 0 ? (0, 3, 6, 9)
                                    // alors on saute la ligne suivante.
        nombre(i * 2, 10, i, 1); // Écrit : 1 2 . 4 5 . 7 8 (pas de 0, 3, 6, 9).
    }
    ecran(1); // On rallume.

    while (true) { // La boucle du jeu.
        image();
    }
}

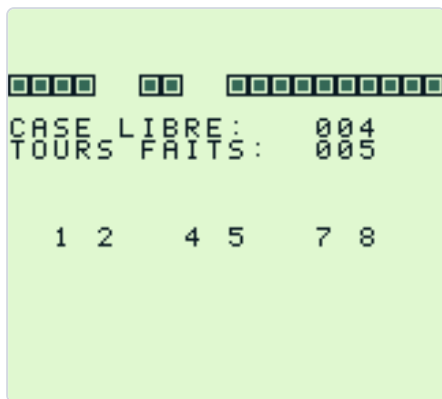
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

La case libre est la 004, trouvée en 005 tours ; puis 1 2 4 5 7 8, sans 0, 3, 6 ni 9.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la première case vide est la colonne 4
- ✓ `break` a arrêté la boucle après 5 tours
- ✓ `continue` a sauté 0, 3, 6 et 9

Ensuite : **32. `do ... while` : au moins une fois** — chapitre 5 : Les boucles : répéter. · [gameboy3](#) — écrire une cartouche Game Boy en C++